

LexiPic

Core Plugin — Technical Documentation

Version: 1.2.2

Type: WordPress plugin (text domain: lexipic)

Requires: WordPress 5.8+, PHP 7.4+

Publisher: BhoomiTech Heritage and Development Foundation

Part of: LexiPic Suite — see "LexiPic Suite Overview" for cross-module context

Table of Contents

Table of Contents.....	2
1. Introduction & Scope.....	4
1.1 Key capabilities	4
2. System Requirements.....	5
3. Architecture Overview	6
3.1 File structure	6
3.2 Component responsibilities	6
3.3 Request lifecycle	7
4. Database Schema	8
4.1 wp_lexipic_sets	8
4.2 wp_lexipic_entries	8
5. REST API Reference	10
5.1 Endpoint summary	10
5.2 Sets endpoints.....	10
GET /sets	10
POST /sets.....	11
DELETE /sets/{id}	11
5.3 Entries endpoints.....	11
GET /sets/{set_id}/entries.....	11
POST /sets/{set_id}/entries	11
5.4 Media streaming endpoints	11
GET /entries/{id}/image.....	11
GET /entries/{id}/audio.....	11
5.5 Export / Import.....	11
GET /sets/{id}/export.....	11
POST /import	12
6. The CompLing AI Transliteration Engine.....	13
6.1 Design philosophy	13
6.2 The four engine files.....	13
Engine settings file.....	14
6.3 Algorithm overview	14
6.4 Snippet hook contract.....	14
7. Language Registry & IME File Management.....	16
7.1 Default languages	16
7.2 Adding a language	16
7.3 Active file resolution	16

7.4 Replacing a file	16
7.5 Backups, restore, and revert	17
7.6 Storage layout	17
8. Front-End Application Architecture	18
8.1 Shortcode & tab visibility	18
8.2 lexipic-app.js structure.....	18
9. Media Handling & Audio Conversion Pipeline	20
9.1 Images.....	20
9.2 Audio & FFmpeg conversion.....	20
10. Permissions & Access Control.....	21
10.1 Summary	21
10.2 Tab access control	21
10.3 Why two layers (UI + REST)	21
11. Settings Reference	22
12. Installation & Configuration	23
13. Uninstall Behavior	24
14. Operational Notes & Known Limitations	25
15. Changelog.....	26

1. Introduction & Scope

LexiPic is the authoring module of the LexiPic Suite. It is a self-contained WordPress plugin that turns any page or post into a structured archive for documenting a heritage or under-resourced language. Contributors pair a photograph of an object, action, or concept with a spoken recording, the word in its native script, and a Roman transliteration generated live as they type, using a built-in phonetic input method engine.

This document describes the plugin's internal architecture: file structure, database schema, REST API, the transliteration engine, the front-end application, media handling, and the permissions model. It assumes the reader is technically familiar with WordPress plugin development. For how this module fits alongside LexiPic Kiosk and the LexiPic Kiosk PWA, see the companion "LexiPic Suite Overview" document.

1.1 Key capabilities

- A three-tab shortcode front end — Archive (browse), Add Entry (capture), Sets (create/manage/export/import) — with the Add Entry and Sets tabs hidden from anyone not permitted to use them.
- In-browser voice capture via the Web Speech API, paired with MediaRecorder-based audio recording.
- A built-in, deterministic phonetic transliteration engine ("CompLing AI") that converts Roman keystrokes to native script live, and native script back to Roman, driven entirely by per-language JSON data files — no code changes needed to support a new orthography.
- Three languages out of the box (Bishnupriya Manipuri, Assamese, Bengali), each with its own speech-recognition locale; additional languages can be registered and customized entirely from wp-admin.
- Optional background FFmpeg audio conversion (.webm → .mp3) for broader playback compatibility, with automatic graceful fallback when FFmpeg is unavailable.
- Per-tab access control by WordPress username, plus an optional public (logged-out) read-only archive view.
- A self-contained JSON import/export format (images and audio embedded as base64) consumed by both LexiPic Kiosk and the LexiPic Kiosk PWA.
- A full lexipic/v1 REST API namespace — the front end never touches the database directly.
- Clean uninstall: deleting the plugin removes its database tables, options, scheduled events, and uploaded files.

2. System Requirements

Requirement	Value
WordPress	5.8 or later (tested up to 7.0)
PHP	7.4 or later
Browser (contributor-facing)	A modern browser; voice capture requires the Web Speech API (SpeechRecognition / webkitSpeechRecognition), broadest in Chromium-based browsers (Chrome, Edge). Microphone access requires HTTPS.
Optional server binary	FFmpeg, for automatic .webm → .mp3 audio conversion. Detected automatically; the plugin works fully without it.
Optional PHP extension	None required for core operation. The plugin uses only standard WordPress/PHP facilities.

3. Architecture Overview

LexiPic follows a conventional single-plugin-file bootstrap (lexipic.php) that defines constants, requires each component file directly (no autoloader), and wires WordPress hooks. There is no build step — all PHP, CSS, and JavaScript ship as authored.

3.1 File structure

lexipic-plugin/	
lexipic.php	Bootstrap: constants, hooks, shortcode
uninstall.php	Removes tables, options, uploads on delete
readme.txt	
includes/	
class-db.php	Lexipic_DB
class-file-handler.php	Lexipic_File_Handler
class-audio-converter.php	Lexipic_Audio_Converter
class-rest-api.php	Lexipic_REST_API
class-ime-manager.php	Lexipic_IME_Manager
admin/	
settings-page.php	Settings screen
import-export.php	Import/Export screen
ime-manager-page.php	IME Engine screen
public/	
css/style.css	
partials/app.php	Shortcode markup (server-rendered)
js/	
lexipic-app.js	Front-end application logic
ime/	Bundled BPM-default engine + data
engine.js	
forwardMap.json	
reverseMap.json	
defaults.json	
js-engine-settings.default.json	

See §3.2 for what each of these files is responsible for.

3.2 Component responsibilities

Component	Responsibility
Lexipic_DB	Owns the two custom database tables and every query against them: sets and entries CRUD, slug uniqueness, entry/audio updates, cascading deletes.
Lexipic_File_Handler	Base64 / binary conversion for images; writes audio files to the uploads directory; builds the portable export array; parses imported JSON (current and legacy formats).
Lexipic_Audio_Converter	Detects FFmpeg availability (cached 1 hour via transient); schedules and performs background .webm → .mp3 conversion via WP-Cron.
Lexipic_REST_API	Registers and implements every route under lexipic/v1 — sets, entries, image/audio streaming, export, import — plus the can_read / can_write permission callbacks.
Lexipic_IME_Manager	Owns the language registry (lexipic_languages option) and the on-disk

Component	Responsibility
	lifecycle of the four CompLing AI engine files per language: resolving the active file, structural validation, replace/backup/restore/download, language add/remove/clone.
public/js/lexipic-app.js	The entire front-end application: tab switching, set/entry CRUD against the REST API, image capture & client-side resize, speech recognition + audio recording, IME wiring, archive card slider.
public/js/ime/engine.js	The fixed CompLing AI transliteration algorithm itself — loaded once per page and driven by whichever per-language JSON data is currently active.

Design note: Lexipic_IME_Manager and engine.js are deliberately separated — the engine (engine.js) is fixed application code, never regenerated, while the JSON data files it reads are fully data-driven and independently replaceable per language without any code change. The plugin's own readme describes this as "never AI-generated at runtime"; only the optional snippet-hook function bodies layered on top may be authored with AI assistance.

3.3 Request lifecycle

1. A page containing [lexipic] is requested. lexipic_render_shortcode() enqueues assets (lexipic_enqueue_assets()) and renders public/partials/app.php, which computes tab visibility server-side from the current user's login state, admin status, and the tab_access setting.
2. wp_localize_script() injects a lexipicConfig object into the page: REST base URL, a WP REST nonce, the active language's speech-recognition locale, FFmpeg availability, whether the visitor can write, and the active language's resolved IME maps/hooks/settings (already resolved server-side via Lexipic_IME_Manager::read_active_json()).
3. public/js/lexipic-app.js boots, loads the list of sets from GET /lexipic/v1/sets, and wires up all UI interaction. Every further read/write goes through the lexipic/v1 REST namespace — the front end never queries the database directly.
4. Audio uploaded via POST .../entries is stored as a file immediately; if FFmpeg is available, Lexipic_Audio_Converter::schedule_conversion() queues a one-off WP-Cron event roughly 5 seconds later to transcode it to MP3 in the background, after which the entry's audio_path/audio_mime are updated and the original .webm is deleted.

4. Database Schema

LexiPic creates two custom tables on activation (Lexipic_DB::create_tables(), via dbDelta()), entirely separate from WordPress posts/postmeta and from the Media Library.

4.1 wp_lexipic_sets

Column	Type	Notes
id	BIGINT(20) UNSIGNED, PK, AUTO_INCREMENT	
name	VARCHAR(255) NOT NULL	Display name, e.g. "Domestic Animals".
slug	VARCHAR(255) NOT NULL, UNIQUE	Derived from name via sanitize_title(); de-duplicated with a numeric suffix on collision.
language	VARCHAR(10) NOT NULL DEFAULT 'bpm'	Language code, must match a key in the language registry (see §7).
created_by	BIGINT(20) UNSIGNED NOT NULL DEFAULT 0	WordPress user ID of the creator; used for ownership checks on delete.
created_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP	

4.2 wp_lexipic_entries

Column	Type	Notes
id	BIGINT(20) UNSIGNED, PK, AUTO_INCREMENT	
set_id	BIGINT(20) UNSIGNED NOT NULL	Foreign key to wp_lexipic_sets.id (indexed, not DB-enforced).
word_script	VARCHAR(500) NOT NULL DEFAULT "	The word in native script.
word_roman	VARCHAR(500) NOT NULL DEFAULT "	Roman transliteration.
description	TEXT, nullable	Optional note.
image_data	MEDIUMBLOB, nullable	Raw binary image data, stored directly in the database row.
image_mime	VARCHAR(50), nullable	e.g. image/jpeg.
audio_path	VARCHAR(500), nullable	Absolute filesystem path under wp-content/uploads/lexipic/.

Column	Type	Notes
audio_mime	VARCHAR(50), nullable	e.g. audio/webm or audio/mpeg after conversion.
created_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP	

Storage design: Images live as MEDIUMBLOB rows in the database; audio lives as files on disk, referenced by path. List queries (Lexipic_DB::get_entries()) deliberately omit image_data to stay fast — the blob is only fetched for a single entry, when actually serving the image or building an export.

5. REST API Reference

All routes are registered under the namespace `lexipic/v1`. Two permission callbacks gate every route:

- **can_read** — true if the visitor is logged in, or if the site-wide "Public Archive" setting is enabled.
- **can_write** — true only if the visitor is logged in. (Per-tab username restrictions are enforced separately by the front-end template, not by this callback — see §11.)

5.1 Endpoint summary

Method	Route	Permission	Purpose
GET	<code>/sets</code>	<code>can_read</code>	List all sets, each annotated with <code>entry_count</code> .
POST	<code>/sets</code>	<code>can_write</code>	Create a new set. Params: <code>name</code> (required), <code>language</code> (default bpm).
GET	<code>/sets/{id}</code>	<code>can_read</code>	Get one set's metadata.
DELETE	<code>/sets/{id}</code>	<code>can_write + owner/admin</code>	Delete a set and all its entries/audio files.
GET	<code>/sets/{set_id}/entries</code>	<code>can_read</code>	List entries for a set (no image blobs), with <code>image_url</code> / <code>audio_url</code> attached.
POST	<code>/sets/{set_id}/entries</code>	<code>can_write</code>	Create an entry. Body: <code>word_script</code> , <code>word_roman</code> , <code>description</code> , <code>image</code> (data URI), <code>audio</code> (data URI).
DELETE	<code>/entries/{id}</code>	<code>can_write</code>	Delete one entry and its audio file.
GET	<code>/entries/{id}/image</code>	<code>can_read</code>	Stream the entry's image binary with correct Content-Type.
GET	<code>/entries/{id}/audio</code>	<code>can_read</code>	Stream the entry's audio file; supports HTTP byte-range requests for seeking.
GET	<code>/sets/{id}/export</code>	<code>can_write</code>	Download a self-contained JSON export of the set (gzip-compressed when the client supports it).
POST	<code>/import</code>	<code>can_write</code>	Import a JSON export (current or legacy format) as a brand-new set.

5.2 Sets endpoints

GET /sets

Returns every set ordered newest-first, regardless of creator — the Archive tab's set selector must show every set any reader is allowed to see. Each set object is annotated server-side with `entry_count` via `Lexipic_DB::count_entries()`. Ownership/ admin restrictions apply only to write actions, not to this listing.

POST /sets

Creates a set owned by the current user. The slug is derived from name and de-duplicated automatically on collision (Lexipic_DB::unique_slug()).

DELETE /sets/{id}

Only the set's creator or a user with manage_options (WordPress Administrator) may delete a set; any other logged-in user receives a 403 forbidden error. Deletion cascades to every entry in the set and removes each entry's audio file from disk.

5.3 Entries endpoints

GET /sets/{set_id}/entries

Returns every entry for a set without the image binary; each entry is annotated with image_url and audio_url pointing at the streaming endpoints below, so the front end never has to construct media URLs itself.

POST /sets/{set_id}/entries

Accepts word_script, word_roman, description as plain strings, and image / audio as base64 data URIs. The handler:

5. Decodes the image data URI to binary via Lexipic_File_Handler::decode_image() (returns a WP_Error on a malformed URI, in which case the entry is still created without an image).
6. Inserts the entry row first, to obtain an entry ID for the audio filename.
7. Saves the audio data URI to a file under wp-content/uploads/lexipic/ via Lexipic_File_Handler::save_audio(), named audio_{entry_id}_{timestamp}.{ext}.
8. Schedules background MP3 conversion via Lexipic_Audio_Converter::schedule_conversion() if FFmpeg is available.
9. Returns the full entry with image_url / audio_url attached and the raw image_data blob stripped from the response.

5.4 Media streaming endpoints

GET /entries/{id}/image

Streams the stored image binary directly from the database row with the correct Content-Type and a 24-hour Cache-Control header.

GET /entries/{id}/audio

Streams the audio file from disk. Supports HTTP Range requests (Accept-Ranges: bytes, partial 206 responses) so an <audio> element can seek without downloading the whole file. Cache-Control is deliberately short (max-age=30, must-revalidate) because the file behind this stable per-entry URL can change from .webm to .mp3 once the background FFmpeg conversion completes — callers that already hold this URL keep working without needing the entry list reloaded.

5.5 Export / Import

GET /sets/{id}/export

Builds the canonical export payload (see the Suite Overview document, §4.1, for the full shape) via `Lexipic_File_Handler::build_export()` and returns it as a downloadable JSON file named `{slug}_lexipic_{YYYYMMDD}.json`. The response is gzip-compressed automatically when the requesting client advertises gzip support, for a significant size reduction on exports with many embedded images/audio.

POST /import

Accepts a JSON body in either the current plugin format (`{set:{...}, entries:[...]}`) or the legacy flat-array format used by the original standalone PWA prototype. `Lexipic_File_Handler::import_json()` detects the shape automatically:

- Current format: set name/slug/language are read from the set object; entries use `word_script` / `word_roman` field names.
- Legacy format: the entire body is treated as the entries array; a set is auto-named "Imported Set {date}"; legacy field names `heritage` and `transliteration` are mapped onto `word_script` and `word_roman` respectively.

Every imported entry is inserted as a brand-new row (IDs are never preserved across import); a new set is always created — import never merges into an existing set.

6. The CompLing AI Transliteration Engine

LexiPic's defining feature is live, bidirectional transliteration between Roman characters and a heritage language's native script, performed entirely client-side as the contributor types. The implementation (public/js/ime/engine.js, "CompLing AI — BPM Fixed Transliteration Engine", currently versioned 4.2.0 internally) is explicitly a fixed, deterministic algorithm — never AI-generated at runtime. Per-language behaviour comes entirely from data: four JSON files per language that the engine reads and interprets the same way regardless of which language they describe.

6.1 Design philosophy

- The engine itself (engine.js) never changes per language or per site — it is shared application code.
- Everything language-specific lives in four JSON data files, independently swappable per language with no code change.
- Edge cases an orthography needs that the fixed engine can't express as simple key-value maps are handled by optional "snippet hook" JavaScript functions — small, targeted functions that may be authored by a human or generated with AI assistance, and which plug into well-defined extension points (pre / loop / post) without touching the engine itself.

6.2 The four engine files

File	Purpose
forwardMap.json	Roman → Script. Maps typed Roman character sequences to native-script output.
reverseMap.json	Script → Roman. The inverse mapping, used to auto-fill the Roman field when script is entered directly (e.g. via speech recognition).
defaults.json	Snippet hook definitions — an array of hook objects (see §6.4) layered on top of the fixed engine for edge cases.
js-engine-settings.default.json	Engine behaviour settings: virama/ZWJ/ZWNJ characters, the inherent vowel token, section processing order, and four hardcoded-rule toggle flags.

Both map files share the same internal section structure, each section being a list of [fromToken, toOutput] pairs:

```
{
  "independent_vowels": [ [...], [...] ],
  "consonants": [
    ["kh", "<consonant glyph>"], ["gh", "<consonant glyph>"], ...
  ],
  "matras": [ [...], [...] ],
  "special": [ [...], [...] ],
  "digits": [
    ["0", "<digit glyph>"], ["1", "<digit glyph>"], ...
  ],
  "punctuation": [ [...], [...] ]
}
```

Lexipic_IME_Manager::REQUIRED_MAP_SECTIONS lists these six section names; a missing section produces a non-blocking warning on upload (the engine simply skips it) rather than a hard failure.

Engine settings file

js-engine-settings.default.json supplies the characters and toggles the algorithm needs but that aren't themselves part of a Roman → Script mapping:

```
{
  "virama": "<virama character>",
  "zwj": "<zero-width joiner>",
  "zwnj": "<zero-width non-joiner>",
  "inherent_vowel": "o",
  "non_linking_consonants": ["ng", "M", "NG", "Ng", ":", "H", "^", "~"],
  "hardcode_independent_vowel": true,
  "hardcode_explicit_zwnj": true,
  "hardcode_consonant_matra": true,
  "hardcode_consonant_cluster": true,
  "section_order": [
    "special", "consonants", "matras",
    "independent_vowels", "digits", "punctuation"
  ]
}
```

The four `hardcode_*` flags toggle built-in structural rules (word-start independent vowel handling, apostrophe → ZWNJ, consonant+matra pairing, consonant-cluster halanta/ZWJ joining). Each can be disabled in favour of a custom loop-stage snippet hook for a language whose orthography needs different behaviour at that step.

6.3 Algorithm overview

10. All sections of the active direction's map are flattened into one lookup list, ordered by the engine settings' `section_order` and, within ties, by longest source key first — so e.g. a three-character cluster is matched before a shorter prefix of the same characters would be.
11. The input string is walked left to right. At each position, pre-stage hooks run first, then the longest matching map entry is applied (unless a loop-stage hook has already "handled" — claimed — that position), then post-stage hooks run.
12. Structural hardcoded rules (independent-vowel-at-word-start, explicit ZWNJ via apostrophe, consonant+matra pairing, consonant-cluster joining via virama/ZWJ) are applied automatically unless disabled in engine settings, in which case a custom hook is expected to take over that responsibility.
13. The result is returned as `{ output, log }` — output is the transliterated string; log entries (if any hook pushed to `state.log`) are available for diagnostics but are not surfaced in the current front-end UI.

6.4 Snippet hook contract

A snippet hook is a single named JavaScript function, stored as a string in `defaults.json` and compiled on demand in the browser via `new Function(...)`. Each hook declares a direction (forward/reverse), a stage (pre/loop/post), an active flag, and a sort order controlling execution sequence relative to other hooks in the same stage:

```
{
  "snippet_key": "fwd_loop_vowel_after_vowel",
```

```

"label": "Subsequent Vowel Independent Form",
"direction": "forward",
"hook_stage": "loop",
"sort_order": 50,
"is_active": 1,
"source": "custom",
"logic_description": "Human-readable explanation shown in the admin UI.",
"js_body": "function fwdLoopVowelAfterVowel(state, config) { /* ... */ }"
}

```

Hook functions receive two arguments:

Parameter	Shape
state	input (full original string), output (accumulated so far), pos (current index, loop stage only), prev (metadata about the previous token), log (array hooks may push diagnostic strings to), handled (boolean — set true in a loop hook to claim the current position and suppress the default map match).
config	maps (active forward/reverse maps), virama, inherent_vowel, direction, consonants_set, matras_set (precomputed Set objects for the active direction), and the section_order / hardcode_* values from engine settings.

Front-end wiring: public/js/lexipic-app.js compiles only active hooks matching the requested direction into pre/loop/post buckets once per page load (getHooksForDirection()), sorted by sort_order — see §8.2.

7. Language Registry & IME File Management

LexiPic → IME Engine (admin/ime-manager-page.php, backed by `Lexipic_IME_Manager`) is where an administrator manages which languages exist and which engine data files are active for each.

7.1 Default languages

Code	Label	Speech locale	Built-in
bpm	Bishnupriya Manipuri	bn-IN	Yes
as	Assamese	as-IN	Yes
bn	Bengali	bn-IN	Yes

This registry is stored in the `lexipic_languages` WordPress option, seeded automatically with the above on first access (`Lexipic_IME_Manager::get_languages()`). Built-in languages cannot be removed via the admin UI (`delete_language()` rejects it), though their engine files can still be replaced like any other language.

7.2 Adding a language

From the "Add a New Language" form, an administrator supplies:

- **Code** — 2–20 lowercase letters/digits/underscores, starting with a letter; validated by regex and rejected on collision with an existing code.
- **Label** — the human-readable display name shown throughout the UI.
- **Speech locale** — a BCP-47 locale for the Web Speech API; defaults to `bn-IN` if left blank.
- **Starting engine files** — either the bundled BPM defaults (the new language has no override files yet and inherits BPM's behaviour until customized), or a clone of another language's currently-active files as a starting point.

Cloning (`Lexipic_IME_Manager::clone_files()`) copies whichever files are currently active for the source language — its own overrides if it has them, or the bundled BPM defaults otherwise — into the new language's storage folder. Each of the four files can then be replaced independently afterward.

7.3 Active file resolution

For a given language and file key, `Lexipic_IME_Manager::active_file_path()` resolves which file is actually read at runtime:

14. If a saved override exists at `wp-content/uploads/lexipic/ime/{lang}/{filename}`, that file is used.
15. Otherwise, the plugin's bundled BPM default at `public/js/ime/{filename}` is used.

This means every newly added language with no customization yet behaves identically to Bishnupriya Manipuri until an administrator uploads replacement files for it — there is no separate "empty" or broken state.

7.4 Replacing a file

`Lexipic_IME_Manager::replace_file()` is the single entry point for uploading a new version of any of the four files, and performs, in order:

16. Validates the upload exists, is under the 5 MB size limit, and has a .json extension (if a filename extension was supplied at all).
17. Parses the file as JSON; a parse failure aborts with the underlying JSON error message and nothing is written.
18. Runs a structural sanity check (`validate_structure()`) appropriate to the file key — e.g. forward/reverse maps are checked for the six required sections, snippet hooks for required fields per entry, settings for expected keys. Problems are returned as non-blocking warnings; the engine tolerates missing sections by skipping them, so the save proceeds regardless.
19. Snapshots whatever file is currently active (override or bundled fallback) into the language's backups/ folder before overwriting it.
20. Re-encodes the decoded JSON (rather than writing the raw uploaded bytes) to normalize formatting and strip any stray BOM or control characters, then writes it to the language's override path.

7.5 Backups, restore, and revert

- Every replace operation automatically backs up whatever was previously active, timestamped as `{key}_{YYYYMMDD-HHMMSS}.json`, capped at the 20 most recent backups per file (older ones pruned automatically).
- "Revert to Default" removes a language's override for one file (after backing it up), causing `active_file_path()` to fall back to the bundled BPM default again.
- "Restore" from the backups list copies a chosen backup back over the active file — the current state is itself backed up first as a safety net before the restore is applied.
- Any individual file, or all four bundled together as a zip (requires the PHP ZipArchive extension), can be downloaded directly from the admin screen for an administrator's own external backup.

7.6 Storage layout

```
wp-content/uploads/lexipic/ime/  
└─ {language-code}/  
    └─ forwardMap.json  
    └─ reverseMap.json  
    └─ defaults.json  
    └─ js-engine-settings.default.json  
    └─ backups/  
        └─ {key}_{YYYYMMDD-HHMMSS}.json (up to 20 most recent per file)
```

8. Front-End Application Architecture

8.1 Shortcode & tab visibility

The `[lexipic]` shortcode accepts an optional `set="{slug}"` attribute to pre-load a specific set, and renders `public/partials/app.php`. Tab visibility is computed entirely server-side, before any markup is sent:

Tab	Visible when
Archive	Always rendered; its content is shown only if the visitor is logged in or the "Public Archive" setting is on. Logged-out visitors without public view see a login prompt instead.
Add Entry	Visitor is logged in AND (is a WordPress Administrator OR their user ID is in the <code>tab_access.add_entry</code> allowlist).
Sets	Visitor is logged in AND (is a WordPress Administrator OR their user ID is in the <code>tab_access.sets</code> allowlist).

If only one tab would be visible, the tab navigation bar itself is omitted entirely (`$show_tab_nav = count($tabs) > 1`) rather than showing a single, non-functional tab button.

8.2 lexipic-app.js structure

The front-end script is a single IIFE with no external dependencies beyond the IME engine. Configuration arrives via `window.lexipicConfig` (injected by `wp_localize_script` — see §3.3). Major responsibilities:

Area	Behaviour
REST client (<code>apiFetch()</code>)	Thin <code>fetch()</code> wrapper that attaches the WP REST nonce, JSON-encodes object bodies, and throws a normalized <code>Error</code> on any non-2xx response.
Tabs	Pure DOM class/attribute toggling between panels; no routing.
Sets	Loads the set list into the selector on boot; pre-selects a set if the shortcode's <code>set=""</code> attribute matches an option's slug; create/delete operate against the REST API and refresh the selector afterward.
Image capture	A selected file is read via <code>FileReader</code> , drawn to an off-screen <code><canvas></code> scaled so neither dimension exceeds 400px (never scaled up), then re-encoded as a JPEG data URI (quality 0.7) for upload — keeping payload size and database row size bounded regardless of the original photo's resolution.
Voice capture	On clicking "Speak", the Web Speech API (<code>SpeechRecognition</code>) and <code>MediaRecorder</code> both start simultaneously against the same microphone stream — recognized text fills the script field (triggering reverse transliteration) while <code>MediaRecorder</code> independently captures the raw audio (audio/webm) for upload. The control is disabled with an explanatory tooltip if the browser lacks <code>SpeechRecognition</code> support entirely.
IME wiring	Typing in the script field runs <code>reverseTranslit()</code> (script → Roman) on every keystroke; typing in the Roman field runs <code>forwardTranslit()</code> (Roman → script) and writes the result into both fields, preserving cursor position in the Roman field.
Snippet hooks	<code>getHooksForDirection()</code> partitions <code>cfg.imeHooks</code> into pre/loop/post buckets per

Area	Behaviour
	direction, filtering out inactive hooks, compiling each <code>js_body</code> via <code>new Function()</code> , and sorting by <code>sort_order</code> — computed once per page load and reused for every keystroke.
Save entry	Validates an image, audio recording, and non-empty script word are all present; converts the recorded audio Blob to a base64 data URI; POSTs the entry; resets the form and refreshes the archive view on success.
Archive slider	Cards are rendered from the current set's entries; Play streams the entry's <code>audio_url</code> ; Delete (shown only when the visitor can write) confirms, deletes via REST, and refreshes both the card list and the set selector's entry count.

Security note: All entry text values are HTML-escaped client-side (`escHtml()`) before being interpolated into card markup, and every write action additionally requires a valid logged-in session enforced server-side by the `can_write` REST permission callback — client-side tab hiding is a UX convenience, not the access-control boundary.

9. Media Handling & Audio Conversion Pipeline

9.1 Images

Images are resized client-side before upload (see §8.2), sent as a base64 data URI, decoded server-side by `Lexipic_File_Handler::decode_image()`, and stored as a MEDIUMBLOB directly in the `wp_lexipic_entries.image_data` column — not in the WordPress Media Library, and not as a file on disk. They are served back out through `GET /lexipic/v1/entries/{id}/image`, which streams the blob with a 24-hour cache header.

9.2 Audio & FFmpeg conversion

Recordings are captured in the browser as audio/webm (the format MediaRecorder produces natively in supporting browsers) and stored as files under `wp-content/uploads/lexipic/`, named `audio_{entry_id}_{timestamp}.webm`.

`Lexipic_Audio_Converter` detects FFmpeg availability by attempting `ffmpeg -version` on the server and caching the result in a transient for one hour (`lexipic_ffmpeg_available`), so every page load doesn't need to shell out. If available, a new recording is converted to MP3 in the background:

21. Immediately after upload, `schedule_conversion()` queues a one-off WP-Cron event 5 seconds in the future via `wp_schedule_single_event()`, passing the entry ID and the `.webm` file path.
22. When the cron event fires, `convert()` runs: `ffmpeg -i {input} -vn -ar 44100 -ac 1 -b:a 64k {output.mp3}` (mono, 44.1kHz, 64kbps — sized for spoken word clips, not music).
23. On success, the entry's `audio_path` / `audio_mime` are updated to point at the new `.mp3` file and the original `.webm` is deleted.
24. On failure (FFmpeg errors, missing binary, etc.), nothing changes — the original `.webm` remains the entry's audio and continues to play normally.

Why MP3 conversion matters: `.webm` audio does not play reliably in Safari/iOS. The plugin therefore upgrades recordings to `.mp3` automatically when possible, while remaining fully functional (just with narrower Safari/iOS support) when FFmpeg isn't installed — nothing in the authoring or viewing flow requires FFmpeg.

The Settings page reports live FFmpeg status (✓ detected / ☐ not detected) so an administrator always knows which playback path is currently active.

10. Permissions & Access Control

10.1 Summary

Action	Requirement
View the archive	Logged in, OR the "Public Archive" setting is enabled for logged-out visitors.
Add an entry	Logged in AND (Administrator OR username listed under "Add Entry" tab access).
Create/manage/export/import a Set	Logged in AND (Administrator OR username listed under "Sets" tab access).
Delete a Set	The Set's creator, OR a WordPress Administrator (manage_options) — independent of the Sets tab-access list.
Delete an entry	Any logged-in user with REST write access (can_write) — the front end only shows the control when CAN_WRITE is true for the current visitor.
Manage Settings / IME Engine	WordPress Administrator (manage_options) only.

10.2 Tab access control

Under LexiPic → Settings, an administrator enters comma-separated WordPress login usernames (not display names) per restricted tab. On save, `lexipic_sanitize_settings()` resolves each username to a numeric WordPress user ID (also accepting raw numeric IDs, for values saved by an older version) and stores the resolved ID list. Unresolvable usernames are rejected with an explanatory admin notice and are not saved.

Administrators always have access to every tab regardless of these lists. Everyone else not explicitly listed — including every logged-out visitor — sees only the Archive tab (subject to the Public Archive setting).

10.3 Why two layers (UI + REST)

Tab visibility in `app.php` is a presentation concern, computed once when the page renders. The REST API independently re-checks permissions on every request: `can_write` requires the visitor to be logged in, and set deletion additionally requires the requester to be the set's creator or a WordPress Administrator.

Note that the per-tab allowlists (Add Entry / Sets) control front-end visibility only — they are not separately re-enforced by the entry-creation or set-creation REST endpoints, which accept any logged-in user regardless of tab-access list membership. In deployments where the Sets/Add Entry allowlists must be a hard security boundary rather than a UI convenience, restrict who can hold a WordPress account on the site, since that is the boundary the REST layer actually enforces for those two actions.

11. Settings Reference

All general settings are stored in the single `lexipic_settings` WordPress option (array), managed from LexiPic → Settings:

Key	Type	Default	Description
language	string	'bpm'	Site-wide default language — sets the speech-recognition locale and IME behaviour for the <code>[lexipic]</code> shortcode.
public_view	0 / 1	0	When enabled, logged-out visitors may browse the Archive tab (still read-only).
tab_access.add_entry	int[] (user IDs)	[]	WordPress user IDs permitted to see/use the Add Entry tab, resolved from usernames on save.
tab_access.sets	int[] (user IDs)	[]	WordPress user IDs permitted to see/use the Sets tab.

The language registry itself (codes, labels, speech locales, built-in flags) is stored separately in the `lexipic_languages` option — see §7.1.

12. Installation & Configuration

25. Upload the lexipic-plugin folder to wp-content/plugins/, or install the zip directly via Plugins → Add New → Upload Plugin.
26. Activate the plugin. On activation, Lexipic_DB::create_tables() creates the two database tables, and lexipic_create_upload_dir() creates wp-content/uploads/lexipic/ (with an .htaccess Options -Indexes for protection) and its ime/ subfolder.
27. Go to LexiPic → Settings: choose the default language, decide whether the archive should be publicly viewable, and optionally configure per-tab username access.
28. Add the shortcode [lexipic] to any page or post. Use [lexipic set="some-slug"] to pre-load a specific set on that page.
29. (Optional) Visit LexiPic → IME Engine to add additional languages or replace the bundled transliteration files for an existing one.

No build step: All assets ship pre-built (plain JS/CSS); there is nothing to compile after installation.

13. Uninstall Behavior

Deleting (not merely deactivating) the plugin from Plugins → Installed Plugins runs `uninstall.php` unconditionally, which:

- Drops both custom database tables (`wp_lexipic_entries`, `wp_lexipic_sets`).
- Deletes the `lexipic_settings` and `lexipic_languages` options, and the `lexipic_ffmpeg_available` transient.
- Recursively deletes the entire `wp-content/uploads/lexipic/` directory — every stored image-adjacent file, audio recording, and custom IME engine file/backup.
- Clears any pending `lexipic_convert_audio` scheduled cron events.

Warning — irreversible: Unlike LexiPic Kiosk (which gates deletion behind an explicit "delete data on uninstall" checkbox), LexiPic's uninstall is unconditional. Administrators should export any Sets they wish to keep before deleting the plugin.

14. Operational Notes & Known Limitations

- Voice capture requires a secure context (HTTPS) — browsers refuse microphone access on plain HTTP except on localhost.
- Web Speech API support varies by browser; Chromium-based browsers have the broadest support. On unsupported browsers the Speak control is disabled with an explanatory tooltip, but script/Roman fields can still be filled manually with full IME support.
- The REST write permission (`can_write`) only checks "is the visitor logged in" — it does not independently enforce the Sets/Add Entry tab-access lists at the API layer (see §10.3). Treat tab access as a UI convenience layered on top of WordPress's own user-account boundary, not a substitute for it.
- Images are stored as MEDIUMBLOB rows (max ~16MB per value, far above what client-side resizing to 400px/JPEG quality 0.7 produces in practice) — very large source photos are always downscaled before upload, so this ceiling is not expected to be reached in normal use.
- Without FFmpeg, audio remains in .webm format indefinitely; this is fully functional in most modern browsers but may not play in Safari/iOS.

15. Changelog

Version	Notes
1.2.2	Current release.

Refer to the plugin's `readme.txt` for the authoritative, up-to-date changelog as future versions are released.